Furthermore, moving away from the partial double allows us to improve the test, too. Maintaining the Arrange/Act/Assert flow helps to keep our tests easy to follow when we return to them months later. Let's convert the double to a spy so we can restore this flow:

15-using-test-doubles-effectively/05/daily_summary_email/spec/daily_summary_spec.rb

```
it "sends a summary of today's messages and threads" do
  email_sender = instance_spy(EmailSender)
  summary = DailySummary.new(email_sender: email_sender)

  summary.send_daily_summary('user@example.com', todays_messages)

  expect(email_sender).to have_received(:deliver).with(
    hash_including(body: 'You missed 3 messages in 2 threads today')
  )
end
```

While we could have used the partial double as a spy, it's more cumbersome, because we would have had to `allow` the `deliver` message beforehand to spy on it and stub it out. Moving to a pure double allowed us to use it as a spy with little effort.

Using Partial Doubles Effectively

Partial doubles are *really* easy to use: just stub or expect a message on any object! However, we said in *Partial Doubles*, on page 241 that we consider their usage to be a code smell. We'd like to flesh that statement out a bit now.

Most unit tests involve a mixture of two types of objects:

- Real objects: typically the subject of the example
- Fake objects: collaborating test doubles used to construct an environment for the test subject

Partial doubles don't fit neatly in this hierarchy. They are *partially* real and *partially* fake. Are they part of what you are testing or part of the environment you are constructing? When an object's roles are unclear, your tests can be harder to reason about.

We prefer not to mix these roles in the same object. In some cases, it can have some serious consequences. Let's look at an example.

Many Software as a Service (SaaS) apps use a monthly subscription model, where customers are charged every month. Here's a typical implementation, using a hypothetical billing API called `CashCow`: